

Time-Off Microservice — Technical Requirements Document

Technical Requirements Document

Time-Off Microservice — ReadyOn

Author: Richard Chen **Status:** Final **Stack:** NestJS 11 · TypeORM · SQLite · Jest

1. Overview

ReadyOn is the employee-facing interface for time-off requests. The Human Capital Management system (HCM — e.g. Workday, SAP) is the authoritative source of truth for employment and balance data.

This microservice sits between the two. It manages the full lifecycle of a time-off request, maintains a local balance cache, syncs with HCM via realtime and batch APIs, and enforces balance integrity even when HCM behaves unreliably.

2. Core Challenges

2.1 HCM Is Not the Only Writer

ReadyOn is not the only system that modifies HCM balances. Payroll events, work anniversaries, and year-start accrual resets all update balances directly in HCM without notifying ReadyOn. This means ReadyOn's local balance copy can become silently stale at any time.

Implication: ReadyOn cannot rely solely on its local balance at the moment of approval. A realtime HCM check is required before every submission.

2.2 HCM Validation Is Unreliable

The HCM spec says it *usually* rejects requests with invalid dimensions or insufficient balances — but this is not guaranteed. Some HCM configurations accept submissions they should reject, potentially allowing balances to go negative in the source of truth.

Implication: ReadyOn must validate defensively before calling HCM, not after. HCM rejection cannot be the only safety net.

2.3 Two Sync Modes With Different Guarantees

HCM exposes two sync surfaces: - **Realtime API:** per-employee-location, low latency, triggered on demand - **Batch endpoint:** full corpus, periodic, authoritative at scale

Neither alone is sufficient. Realtime is too slow for bulk reconciliation. Batch is too infrequent for pre-submission freshness.

Implication: Both must be supported and used at the right moments.

2.4 Concurrency and Idempotency

Employees may submit duplicate requests (client retries, double-click). Multiple concurrent requests for the same employee-location pair may race. A failed HCM call may be retried, risking double deduction.

Implication: The request flow must be serialized per employee-location and must support idempotent retries.

3. Data Model

Employee	Location	EmployeeLocation
id (uuid) name status ACTIVE INACTIVE	id (uuid) name ...	id (uuid) employeeId → Employee locationId → Location active: boolean UNIQUE(employeeId, locationId)
TimeOffBalance		TimeOffRequest
id (uuid) employeeId → Employee locationId → Location balanceDays: DECIMAL(10,2) lastSyncedAt: datetime source: HCM_BATCH HCM_REALTIME LOCAL_PENDING missingFromLatestBatch: boolean UNIQUE(employeeId, locationId)		id (uuid) employeeId → Employee locationId → Location requestedDays: DECIMAL(10,2) status: PENDING APPROVED APPROVED_WITH_WARNING REJECTED FAILED failureReason: text? hcmTransactionId: varchar? idempotencyKey: varchar? createdAt, updatedAt

Key decisions: - DECIMAL(10,2) for balances — supports half-days, avoids float imprecision - missingFromLatestBatch — enables safe mark-and-sweep without hard deletes - APPROVED_WITH_WARNING status — captures HCM successes with no transaction ID for audit - idempotencyKey — enables safe client retries without duplicate state

4. API Design

Balance Endpoints

Method	Endpoint	Description
GET	/balances/:employeeId	All balances across locations
GET	/balances/:employeeId/:locationId?refresh=true	Single balance; fetches from HCM realtime if refresh=true

Time-Off Request Endpoints

Method	Endpoint	Description
POST	/time-off-requests	Submit a request (body: employeeId, locationId, requestedDays,

Method	Endpoint	Description
GET	/time-off-requests/:id	Request by ID
GET	/time-off-requests? employeeId=&locationId=&status=	Filtered list

Sync Endpoints

Method	Endpoint	Description
POST	/sync/hcm/batch	Ingest full HCM balance corpus
POST	/sync/hcm/reconcile/:employeeId/:locationId	Force realtime reconciliation for one pair

5. Time-Off Request Flow

Every POST /time-off-requests follows a deterministic 14-step flow:

Phase 1 — Local validation (fast, no external calls)

1. requestedDays > 0 — reject immediately if not
2. Employee exists and is ACTIVE
3. Location exists
4. EmployeeLocation pairing exists and is active
5. Idempotency key check — if a request with this key already exists, return it immediately (no reprocessing)

Phase 2 — Concurrency control + balance check

6. Acquire async-mutex lock keyed on (employeeId, locationId) — serializes concurrent requests for the same pair
7. Load local balance — reject with InsufficientBalanceException if clearly insufficient (avoids unnecessary HCM call)

Phase 3 — HCM realtime validation

8. Fetch realtime balance from HCM
9. If HCM times out → create FAILED request, release lock, return error (no deduction)
10. If HCM balance differs from local → update local balance (source: HCM_REALTIME)
11. Re-check balance against updated HCM value — reject if still insufficient

Phase 4 — Submission (inside DB transaction)

12. Submit deduction to HCM (POST /hcm/time-off)
13. If HCM times out or returns error → create FAILED request, do not deduct locally
14. If HCM returns success:
 - Deduct local balance atomically with request creation
 - If transaction ID present → APPROVED
 - If no transaction ID → APPROVED_WITH_WARNING (still deduct; flag for audit)

6. HCM Sync Strategy

Realtime Reconciliation

Used in two places: - Pre-submission (step 8 above) — every request fetches fresh HCM balance before approval
 - POST /sync/hcm/reconcile/:emp/:loc — force-refresh a single pair on demand

Source tag: HCM_REALTIME

Batch Sync

POST /sync/hcm/batch accepts the full HCM balance corpus and upserts locally using a **mark-and-sweep** algorithm:

1. Begin transaction
2. Mark all existing TimeOffBalance rows `missingFromLatestBatch = true`
3. For each record in the payload:
 - **Negative balance** → quarantine (skip, log warning, increment quarantined counter)
 - **Duplicate (same emp+loc)** → last-wins by payload order, log conflict, increment skipped
 - **Otherwise** → upsert with `missingFromLatestBatch = false`, source: HCM_BATCH
4. Commit
5. Count rows still marked `missingFromLatestBatch = true` → report as `stillMissing`

Returns: { updated, inserted, skipped, quarantined, stillMissing }

Why mark-and-sweep and not delete? A missing pair in a batch payload could be a data gap or a temporary omission. Hard-deleting would destroy history and could mask bugs. Flagging preserves the record and allows operators to investigate.

Source tag: HCM_BATCH

7. Failure Modes and Handling

Scenario	Outcome	Local balance
HCM timeout during balance fetch (step 9)	FAILED	Unchanged
HCM timeout during submit (step 13)	FAILED	Unchanged
HCM 500 during submit	FAILED	Unchanged
HCM returns success, no transaction ID	APPROVED_WITH_WARNING	Deducted
HCM rejects (insufficient / bad dimensions)	FAILED	Unchanged
HCM in <code>unreliableValidation</code> mode accepts bad request	Caught at step 7 or 11 — never reaches HCM	Unchanged
Batch sync HCM server error	Entire batch rolled back	Unchanged
Batch record has negative balance	Quarantined, skipped	Unchanged

Invariant: The local balance can never go below zero under any code path. Both the local pre-check and the post-HCM re-check enforce this before the DB transaction is opened.

8. Concurrency and Idempotency

Concurrency

SQLite does not support `SELECT FOR UPDATE`. To prevent the read-check-write race condition (two concurrent requests both read a 10-day balance, both pass the check, both deduct), the service maintains a `Map<string, Mutex>` keyed on `"${employeeId}:${locationId}"` using the `async-mutex` library.

Within the mutex, the sequence is: read balance → validate → call HCM → commit in transaction. The entire critical section is serialized.

Tradeoff: This is correct and fast for a single-instance deployment. For horizontal scaling, this would need to be replaced with a distributed lock (Redis Redlock, database advisory lock, or queue-based serialization). Documented in README.

Idempotency

An optional `idempotencyKey` field on `POST /time-off-requests` enables safe client retries. The key is stored on the `TimeOffRequest` row. On receipt, before acquiring the mutex, the service checks for an existing request with that key. If found, it returns the original result immediately — no reprocessing, no risk of double deduction.

9. Alternatives Considered

GraphQL vs REST

GraphQL would allow clients to request exactly the fields they need (useful for the balance + request combo view). However, the spec described clear, resource-oriented operations with no complex nested query requirements. REST keeps the implementation simple, observable, and cacheable. Chosen: **REST**.

Postgres vs SQLite

Postgres offers `SELECT FOR UPDATE` (eliminating the need for application-level locking), `LISTEN/NOTIFY` for event-driven reconciliation, and better horizontal scaling. However, the spec called for SQLite explicitly, and for a single-instance microservice the tradeoff is acceptable. The mutex-based locking fills the gap. If this service needed to scale horizontally, the migration path to Postgres is straightforward — TypeORM supports both.

Soft lock (optimistic) vs Mutex (pessimistic)

Optimistic concurrency (retry on version mismatch) would avoid blocking concurrent requests for different operations. However, time-off requests are inherently low-frequency and high-stakes — a blocked request for 100ms is far preferable to a failed retry cycle that leaves the user uncertain. **Pessimistic locking (mutex)** chosen for predictability.

Hard delete vs mark-and-sweep for batch sync

Hard-deleting pairs missing from a batch would keep the local DB clean. But it risks destroying valid records if the batch payload has a data gap or was truncated mid-transfer. Mark-and-sweep preserves history, allows investigation, and is reversible. **Mark-and-sweep** chosen.

APPROVED_WITH_WARNING vs FAILED (no transaction ID)

Two reasonable choices: treat a missing transaction ID as a failure (safe but may alarm users on valid approvals), or treat it as a warning (approved but flagged). HCM accepted the request and deducted the balance on its side; rejecting locally would create a split-brain state. **APPROVED_WITH_WARNING** chosen — balance deducted locally, flagged for audit.

10. Test Strategy

Philosophy: Tests as the Specification

The take-home brief calls this out directly: *“since you are using Agentic Development, the value of your work lies in the rigor of your tests.”* When an AI agent writes the implementation, the tests are the only human-authored proof that the system behaves correctly. They are also the regression guard — any future change (by an agent or a human) that silently breaks behavior will break a test.

The test suite was written to mirror the spec exactly: every stated challenge has one or more named tests. Tests are deterministic, isolated, and run in-process with no external dependencies.

Test Architecture

Unit Tests — 43 tests (test/unit/)

All external dependencies are mocked via Jest. Each test exercises a single service method in isolation.

Coverage includes: - TimeOffService: all 14 request flow steps, including each failure mode - SyncService: mark-and-sweep logic, deduplication (last-wins), negative balance quarantine, reconcile path - BalancesService: local-only read vs. HCM refresh path - HcmClientService: correct error type thrown for timeout (HcmTimeoutError), 500 (HcmServerError), 404 (HcmNotFoundError)

Value: Millisecond feedback. Safe to run on every commit. Isolates exactly which branch or condition failed.

Integration Tests — 21 tests (test/integration/)

Real in-memory SQLite database (better-sqlite3, :memory:). HcmClientService is overridden via NestJS DI with MockHcmClientService — no HTTP calls, but the same typed error surface.

Coverage includes: - Full DB round-trips: insert, read, upsert, transaction rollback - Concurrency: two Promise.all requests for overlapping days — exactly one APPROVED, one InsufficientBalanceException - Idempotency key deduplication verified at the DB level - Batch sync mark-and-sweep verified against real SQLite rows

Value: Catches TypeORM quirks (e.g. delete({}) vs clear(), update({}) vs QueryBuilder) and schema-level constraints that mocks paper over. Guards against regressions from ORM version upgrades.

E2E Tests — 36 tests (test/e2e/)

Full NestJS HTTP application via supertest, in-memory SQLite, MockHcmClientService injected via DI. Every test hits the HTTP layer and asserts on the HTTP response body and DB state.

All 30 spec scenarios are covered as named tests: - **Happy paths** (1–4): approval, balance read, batch insert, batch update - **Validation edge cases** (5–11): zero days, negative days, nonexistent employee/location, inactive employee, inactive pairing, insufficient local balance - **HCM staleness** (12–16): stale local (HCM has more),

stale local (HCM has less), work anniversary, year-start refresh - **Defensive HCM behavior** (17–21): unreliable validation mode, missing transaction ID, timeout on balance check, timeout on submit, server error during batch

- **Concurrency/idempotency** (22–24): two simultaneous requests, duplicate idempotency key, retry after failure
- **Data integrity** (25–30): new pair inserted, omitted pair flagged, duplicate batch records, negative balance quarantined, decimal balances, no negative local balance

Value: The contract test for the full system. If all 36 pass, the microservice behaves exactly as specified. These are the tests a future developer (or agent) must not break.

Mock HCM Server

MockHcmStore is an in-memory Map<string, number> with a configurable mode field. It is instantiated once per test suite and injected via NestJS DI, replacing the real HcmClientService. This means:

- Zero network calls in any test
- Fully deterministic — no flakiness from timing or network
- Mode switches are synchronous — tests can toggle unreliableValidation → normal → timeout mid-suite
- store.reset() in beforeEach gives each test a clean slate

Modes: normal · unreliableValidation · timeout · serverError · staleResponse

The mock also exposes the full HCM REST API as a NestJS module (MockHcmModule) for use in any deployment that needs a real HTTP mock server, without changing the test DI approach.

Why This Structure

Layer	What it catches	Speed
Unit	Logic errors, wrong branch, missing edge case	~1s total
Integration	ORM behavior, schema constraints, transaction atomicity	~5s total
E2E	Full system correctness, HTTP contract, all spec scenarios	~10s total

No test layer is redundant. Each catches a distinct class of failure. The pyramid (many unit, fewer integration, fewer E2E) is intentional — but every E2E test is load-bearing because it maps 1:1 to a spec requirement.

11. Project Structure

```
src/  
  app.module.ts  
  main.ts  
  common/  
    exceptions/      - typed exceptions (EmployeeNotFound, HcmTimeout, etc.)  
    filters/         - global HTTP exception filter  
  database/  
    database.module.ts - TypeORM + SQLite setup  
    entities/        - all 5 TypeORM entities  
    seeds/           - local development seed data  
  employees/  
  locations/  
  balances/  
  time-off/  
  hcm-client/  
  mock-hcm/  
  sync/             - batch sync + realtime reconcile
```

```
test/
  helpers/      - shared test app factory, seed helper, MockHcmClientService
  unit/         - 4 spec files, all deps mocked
  integration/  - 3 spec files, real SQLite + mock HCM via DI
  e2e/          - 1 spec file, full HTTP stack, all 30 scenarios
```

12. Setup

```
npm install
cp .env.example .env
mkdir -p data
npm run start:seed    # seed local SQLite with employees, locations, balances
npm run start:dev     # dev server on :3000

npm test              # all 100 tests
npm run test:cov      # with coverage report (./coverage/)
npm run build          # compile to ./dist
npm run start:prod    # run compiled build
```

Environment variables:

Variable	Default	Description
DATABASE_PATH	./data/timeoff.db	SQLite file path (use absolute path in production)
HCM_BASE_URL	http://localhost:3001	Real HCM base URL
HCM_TIMEOUT_MS	5000	HCM request timeout in milliseconds
PORT	3000	Service port